## How The Apps Work Together:

A) Simplified to two main apps, logical grouping of related features
B) Users app handles authentication, profiles and user related functionalities
C) Projects app handles everything video-related

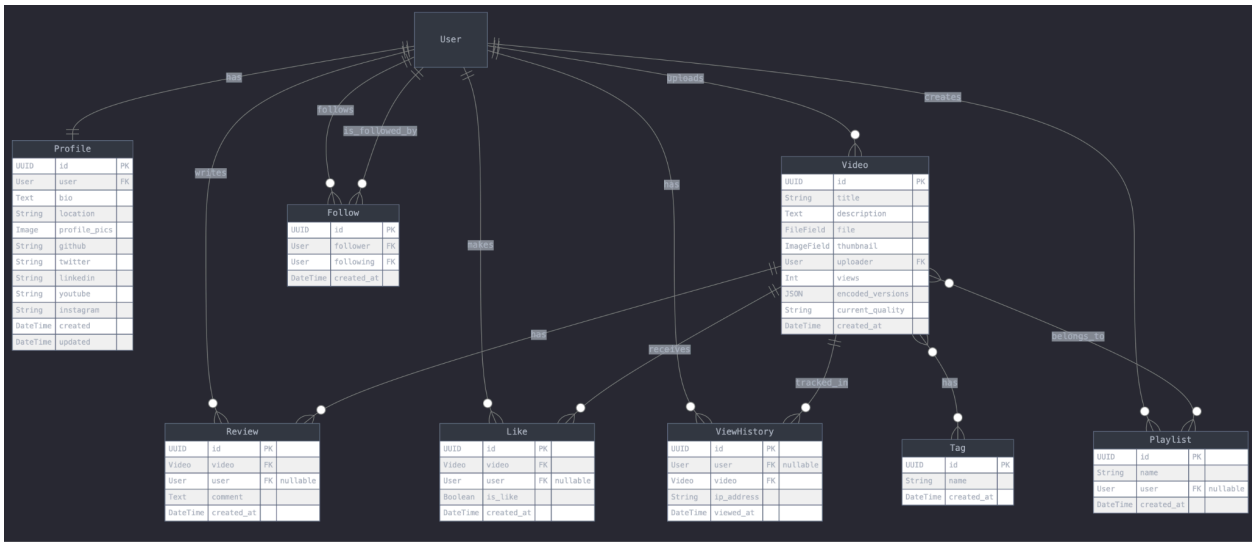
1.”users” App (User Management) (***Key Models-User, Profile, Follow, ViewHistory*** )

| Core Purpose - | Advanced Purpose - |
|---|---|
| 1.Handles user accounts (signup/login)✔️<br>2.Manages user profiles✔️<br>3.Stores user information✔️<br>4.Manages the ‘user to user following’ feature✔️<br>5.Handles Authentication & Authorization ❓<br>6.Core Authentication & Security ❓ | 1.Monetization ⁉️⁉️ |

2.”projects” App (Core Features) (***Key Models-Video, Tag, Like, Review, Playlist )***

| Core Purpose - | Advanced Purpose - |
|---|---|
| 1.Handles video uploads ✔️<br>2.Manages video playback ✔️<br>3.Stores all the relevant video metadata ❓<br>4.Manages video listings ✔️<br>5.Handles video reviews/ratings ✔️<br>6.Manages video tagging system ✔️<br>7.Fitting the thumbnails with Bars ✔️<br>8.Managing user’s viewing history ✔️<br>9.Video Compression and Encoding ✔<br>10. Video analytics page ❓ ❓ | 1.Thumbnail Generation ⁉️<br>2.Ability to Handle Live Streaming ⁉️⁉️<br>3.Security and Compliance ⁉️⁉️<br>4.Creator Tools Page ⁉️⁉️ |

## DB Schema




Key Relationships Between Tables:

| | | |
|---|---|---|
| 1) User - Profile Relationship<br><br>Practical: Each user has exactly one profile, and each profile belongs to exactly one user<br>Technical: One-to-One relationship with Profile.user_id referencing User.id<br>Example: Like a YouTube account (User) having one channel profile (Profile)<br>Access: user.profile or profile.user | 2) User - Video Relationship<br><br>Practical: One user can upload many videos, but each video has one uploader<br>Technical: One-to-Many relationship with Video.uploader_id referencing User.id<br>Example: A YouTuber uploading multiple videos to their channel<br>Access: user.videos.all() or video.uploader | 3) User - Review Relationship<br><br>Practical: One user can write many reviews, each review has one author<br>Technical: One-to-Many relationship with Review.user_id referencing User.id<br>Example: A user commenting on multiple different videos<br>Access: user.reviews.all() or review.user |

| | | |
|---|---|---|
| 4) User - Like Relationship<br><br>Practical: One user can like many videos, each like comes from one user<br>Technical: One-to-Many with Like.user_id referencing User.id<br>Example: A user liking multiple different videos<br>Access: user.video_likes.all() or like.user | 5) User - Playlist Relationship<br><br>Practical: One user can create many playlists, each playlist has one owner<br>Technical: One-to-Many with Playlist.user_id referencing User.id<br>Example: A user creating multiple playlists for different video categories<br>Access: user.playlists.all() or playlist.user | 6) Video - Tag Relationship<br><br>Practical: One video can have many tags, one tag can be on many videos<br>Technical: Many-to-Many relationship using an automatic junction table<br>Example: A video having multiple tags (#funny, #music, #tutorial)<br>Access: video.tags.all() or tag.video_set.all() |
| 7) Video - Playlist Relationship<br><br>Practical: One video can be in many playlists, one playlist can have many videos<br>Technical: Many-to-Many relationship using an automatic junction table<br>Example: Adding a music video to multiple different playlists<br>Access: playlist.videos.all() or video.playlists.all() | 8) User - Follow Relationship<br><br>Practical: Users can follow many users and be followed by many users<br>Technical: Many-to-Many through explicit Follow model with additional data<br>Example: Like following multiple YouTube channels<br>Access: user.following_set.all() or user.followers_set.all() | 9) Video - Like Relationship<br><br>Practical: One video can have many likes, each like belongs to one video<br>Technical: One-to-Many with Like.video_id referencing Video.id<br>Example: Multiple users liking or disliking a single video<br>Access: video.likes.all() or like.video |
| 10) Video - Review Relationship<br><br>Practical: One video can have many comments, each comment belongs to one video<br>Technical: One-to-Many with Review.video_id referencing Video.id<br>Example: Like YouTube comments where multiple users can comment on a single video | 11) Video - ViewHistory Relationship<br><br>Practical: One video can have many view records<br>Technical: One-to-Many with ViewHistory.video_id referencing Video.id<br>Example: Tracking individual views of a video<br>Access: video.views_history.all()<br>Features: Tracks timestamp and IP for anonymous views | 12) User - ViewHistory Relationship<br><br>Practical: Users can have viewing history for multiple videos (nullable)<br>Technical: One-to-Many with ViewHistory.user_id referencing User.id (null=True)<br>Example: Tracking what videos a user has watched<br>Access: user.view_history.all() |

# GITHUB WORKFLOW

| | | |
|---|---|---|
| 1.Initial Status and Feature Branch Work<br><br># Check current branches (*indicates current branch)<br>> git branch<br><br># Stage all changes<br>> git add .<br><br># Commit changes with message<br>> git commit -m "your message"<br><br># Push feature branch to remote<br>> git push origin feature/branch-name | 2. Update Main Branch<br><br># Switch to main branch<br>> git checkout main<br><br># Pull changes from develop to main<br>> git pull origin develop<br><br># Push changes to main<br>> git push origin main<br><br>#If rejected:<br># sync with remote main<br>> git pull origin main<br><br># try pushing again<br>> git push origin main | 3.Update Develop Branch<br><br># Switch to develop<br>> git checkout develop<br><br># Get latest changes<br>> git pull origin develop<br><br># Merge feature branch into develop<br>> git merge feature/branch-name<br><br># Push updated develop<br>> git push origin develop |
| 4.Clean Up Feature Branch<br><br># Delete local feature branch<br>> git branch -d feature/branch-name<br><br># Delete remote feature branch<br>> git push origin --delete feature/branch-name | 5.Start New Feature Branch<br><br># Create and switch to new feature branch from develop<br>> git checkout develop<br>> git checkout -b feature/new-branch-name<br><br># Push new branch to remote and set tracking<br>> git push -u origin feature/new-branch-name | 6.Final Sync<br><br># Get all remote updates<br>> git fetch --all<br><br># Pull updates for all branches<br>> git pull --all<br><br># Verify branch structure<br>> git branch |